
3.7.4 RESUMEN DE SECCIÓN

1. **La comprensión de listas** te permite crear nuevas listas a partir de las existentes de una manera concisa y elegante. La sintaxis de una comprensión de lista es la siguiente:

```
[expresión para el elemento en la lista si es condicional]
```

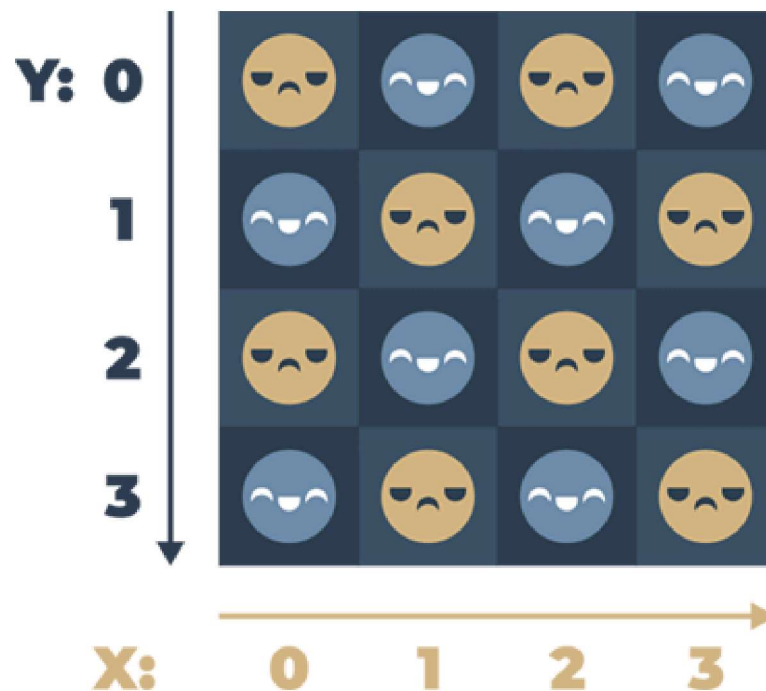
El cual es un equivalente del siguiente código:

```
for element in list:
    if conditional:
        expression
```

Este es un ejemplo de una comprensión de lista - el código siguiente crea una lista de cinco elementos con los primeros cinco números naturales elevados a la potencia de 3:

```
1 | cubed = [num ** 3 for num in range(5)]
2 | print(cubed) # output: [0, 1, 8, 27, 64]
3 |
```

2. Puedes usar **listas anidadas** en Python para crear **matrices** (es decir, listas bidimensionales). Por ejemplo:

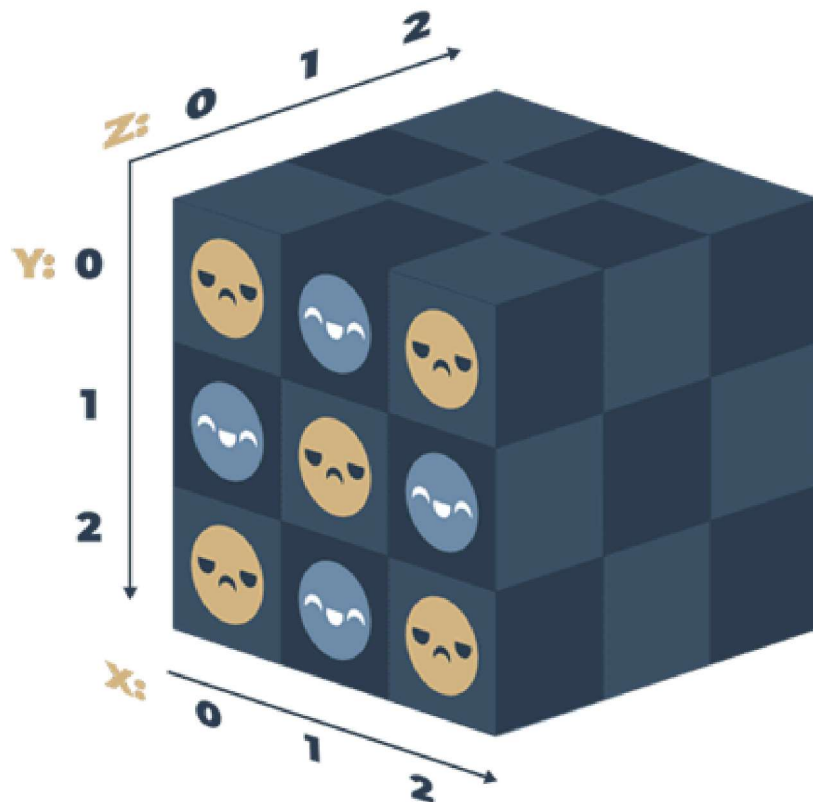


```

1  # Una tabla de cuatro columnas y cuatro filas: un arreglo bidimensional (4x4)
2
3  table = [[:(" ", "):"), ":((" ", "):"),
4            [":(" ", "):"), ":((" ", "):"),
5            [":(" ", "):"), ":((" ", "):"),
6            [":(" ", "):"), ":((" ", "):"),
7
8  print(table)
9  print(table[0][0]) # output: ':(('
10 print(table[0][3]) # output: '):)'
11

```

3. Puedes anidar tantas listas en las listas como desee y, por lo tanto, crear listas n-dimensionales, por ejemplo, arreglos de tres, cuatro o incluso sesenta y cuatro dimensiones. Por ejemplo:



```

1  # Cubo - un arreglo tridimensional (3x3x3)
2
3  cube = [[[':(', 'x', 'x'],
4            [':)', 'x', 'x'],
5            [':(', 'x', 'x']],
6
7            [[':)', 'x', 'x'],
8            [':(', 'x', 'x'],
9            [':)', 'x', 'x']],
10
11           [[':(', 'x', 'x'],
12            [':)', 'x', 'x'],
13            [':)', 'x', 'x']]
14
15  print(cube)
16  print(cube[0][0][0]) # output: ':(('

```

```
17 | print(cube[2][2][0]) # output: ':'  
18 |
```